

Manual de Desarrollo Visual — Dashboard Streamlit

Propósito de este documento: Entender cómo se diseñó y construyó el dashboard interactivo: sistema de diseño, componentes, CSS personalizado, visualizaciones y decisiones de UX.

Índice

- ¿Por qué Streamlit?
- Arquitectura de app.py
- Sistema de diseño — Tokens y tipografía
- Inyección de CSS — Cómo personalizar Streamlit
- Componentes clave
- Navegación — Sidebar como menú
- Las 5 páginas del dashboard
- Visualizaciones — Plotly en Streamlit
- Manejo de datos — Cache y filtros
- Animaciones y micro-interacciones
- Cómo ejecutar y personalizar

1. ¿Por qué Streamlit?

Streamlit es un framework de Python que convierte scripts de datos en aplicaciones web sin necesidad de conocimientos de HTML, CSS o JavaScript.

Ventajas para un portafolio de Data Analyst

Criterio	Streamlit	Flask/FastAPI + HTML	Node.js
Curva de aprendizaje	Baja — Python puro	Media — requiere HTML/Jinja	Alta — nuevo lenguaje
Integración con pandas	Nativa	Manual	Manual
Integración con Plotly	Nativa	Manual	Posible con Plotly.js
Perfil que demuestra	Data Analyst (ideal)	Data Engineer	Full-Stack
Tiempo de desarrollo	Horas	Días	Semanas

Para un DA, lo que importa es demostrar que dominas el dato y puedes visualizarlo con claridad. Streamlit permite eso sin salir del ecosistema Python.

Limitaciones conocidas de Streamlit

- **Re-render completo:** Cada interacción del usuario ejecuta el script de arriba a abajo (mitigado con `@st.cache_data`)
- **CSS limitado:** No se puede acceder directamente a los componentes renderizados — hay que usar CSS hackeando los `data-testid` de los elementos internos
- **No es React:** No hay estado local por componente — todo el estado es global (`st.session_state`)

2. Arquitectura de app.py

Estructura del archivo

```
app.py
|
├─ Constantes globales
|   ├─ Rutas: DB_PATH, CSV_PATH
|   ├─ Tokens de color: ACCENT, BG, CARD_BG, TEXT, TEXT_SUB
|   ├─ TYPE_COLORS: paleta por tipo Pokémon
|   └─ NAV_PAGES: lista de páginas
|
├─ Funciones de configuración
|   ├─ inject_css()      ← Carga fuentes + todo el CSS personalizado
|   └─ load_data()      ← Lee datos de SQLite (fallback a CSV)
|
├─ Funciones de UI por página
|   ├─ tab_overview()   ← Vista General
|   ├─ tab_stats()      ← Estadísticas
|   ├─ tab_gallery()    ← Galería
|   ├─ tab_compare()    ← Comparación
|   └─ tab_etl()        ← Resumen ETL
|
└─ main()
    ├─ st.set_page_config()
    ├─ inject_css()
    ├─ load_data()
    ├─ render_sidebar()  ← Logo + filtros + navegación
    └─ Router: llama a la función según la página seleccionada
```

Flujo de ejecución

```
Usuario interactúa
|
▼
Streamlit re-ejecuta app.py completo
|
├─ inject_css()      ← CSS siempre cargado
├─ load_data()       ← Datos del cache (no re-lee el archivo)
├─ render_sidebar()  ← Filtros aplicados al DataFrame
└─ tab_*( )         ← Renderiza la página seleccionada
```

3. Sistema de diseño — Tokens y tipografía

Tokens de color

Los colores no se hardcodean en cada elemento — se definen como constantes al inicio del archivo y se usan mediante f-strings en el CSS.

```
ACCENT = "#2962FF" # Azul principal - acento, bordes, valores KPI
```

```
BG          = "#F5F7FA"  # Fondo general — gris muy claro
CARD_BG     = "#FFFFFF"  # Fondo de tarjetas — blanco puro
TEXT        = "#111111"  # Texto principal — casi negro
TEXT_SUB    = "#555555"  # Texto secundario — gris medio
```

¿Por qué este sistema? Si se quiere cambiar el color de acento de azul a verde, el cambio es una sola línea: `ACCENT = "#2E7D32"`. Sin tokens, habría que buscar y reemplazar `#2962FF` en 40+ lugares.

Paleta de tipos Pokémon

```
TYPE_COLORS = {
  "normal": "#A8A878", "fire": "#F08030", "water": "#6890F0",
  "electric": "#F8D030", "grass": "#78C850", ...
}
```

Estos colores son los canónicos del universo Pokémon (usados en los juegos y en Bulbapedia). Darles el color correcto no es decorativo — el lector que conoce Pokémon los reconoce inmediatamente, lo que reduce la carga cognitiva del dashboard.

Tipografías

El dashboard usa dos fuentes con propósitos distintos:

```
/* Cargadas desde Google Fonts */
@import 'Inter:wght@400;500;600;700;800'
@import 'Press+Start+2P'
```

Fuente	Dónde se usa	Por qué
Press Start 2P	Brand, logo, títulos de página, nav	Es la fuente "gamer/pixel" — evoca Pokémon directamente
Inter	Todo el cuerpo de texto, datos, labels	Diseñada para pantallas, legible a tamaños pequeños

La regla base es:

```
* { font-family: 'Inter', sans-serif !important; }
.pokemon-brand { font-family: 'Press Start 2P', monospace !important; }
```

Todo usa Inter por defecto. Solo los elementos marcados con `.pokemon-brand` usan Press Start 2P.

Jerarquía tipográfica — Escala establecida

Press Start 2P (escala reducida porque es una fuente muy grande visualmente):

Elemento	Tamaño	Dónde
Brand / logo	<code>`0.85rem`</code>	Header principal
Títulos de página h2	<code>`0.80rem`</code>	<code>` .page-header h2`</code>
Ítems de navegación	<code>`0.58rem`</code>	Sidebar radio buttons
Subtítulo principal	<code>`0.55rem`</code>	<code>` .page-header p`</code>
Labels de sección sidebar	<code>`0.52rem`</code>	<code>`st.caption()`</code>

Inter (escala estándar):

Elemento	Tamaño	Dónde
KPI value	`1.9rem`	` .kpi-value`
Section title	`1.05rem`	` .section-title`
Narrative card title	`0.90rem`	Sección ETL
Nombre en poke-card	`0.82rem`	Galería
Selectbox value	`0.82rem`	Filtro burbuja
KPI label	`0.75rem`	` .kpi-label`
Type badge / bst label	`0.72rem`	Galería
Hero badge	`0.72rem`	Vista General
Winner name (Comparación)	`0.68rem`	Ganador en comparación
Type badge Comparación	`0.68rem`	Comparación
Alert "!" Press Start 2P	`0.80rem`	Comparación
Selectbox label	`0.62rem`	Filtro burbuja
Labels varios	`0.56rem`	Sidebar, detalles

4. Inyección de CSS — Cómo personalizar Streamlit

El problema: Streamlit no expone sus estilos

Streamlit renderiza los componentes con React internamente. No existe un archivo CSS que editar directamente. La única forma de personalizar los estilos es:

```
st.markdown("""<style> ... </style>""", unsafe_allow_html=True)
```

`unsafe_allow_html=True` es necesario porque Streamlit por defecto escapa el HTML para evitar XSS. Al usarlo, asumimos la responsabilidad de que el HTML es seguro.

Cómo apuntar a componentes nativos

Streamlit añade atributos `data-testid` a sus contenedores. Se pueden usar como selectores CSS:

```
/* Sidebar */
[data-testid="stSidebar"] { background: ...; }

/* Gráfica Plotly */
[data-testid="stPlotlyChart"] { border-radius: 20px; }

/* Selectbox completo (label + dropdown) */
.main div[data-testid="stSelectbox"] { background: ...; }
```

Importante: Los `data-testid` pueden cambiar entre versiones de Streamlit. Si el CSS deja de funcionar tras una actualización, lo primero es inspeccionar el DOM y verificar que el atributo sigue siendo el mismo.

Scoping: `.main` vs `[data-testid="stSidebar"]`

```
/* CORRECTO: solo afecta selectboxes en el área principal */
.main div[data-testid="stSelectbox"] { ... }
```

```
/* INCORRECTO: afectaría también los selectboxes del sidebar */
div[data-testid="stSelectbox"] { ... }
```

El selector `.main` apunta al contenedor principal (excluyendo el sidebar). Esto permite aplicar estilos diferentes al mismo componente dependiendo de dónde esté ubicado.

Por qué `st.markdown('<div>') + widget + st.markdown('</div>')` NO funciona

Una confusión común al trabajar con Streamlit CSS es intentar envolver widgets en divs HTML:

```
st.markdown('<div class="mi-contenedor">')
st.selectbox("Filtro", options=[...])  # ← Este widget NO queda dentro del div
st.markdown('</div>')
```

Cada llamada a `st.markdown()` y `st.selectbox()` genera un elemento DOM separado. No se pueden anidar de esta forma. La única solución para aplicar estilos a un widget nativo es apuntar directamente a su `data-testid` con CSS global.

El filtro burbuja — Selectbox con estilo de card

```
.main div[data-testid="stSelectbox"] {
  background: #FFFFFF;
  border: 1.5px solid #2962FF33;          /* ACCENT con 20% de opacidad */
  border-radius: 16px;
  padding: 0.3rem 0.75rem 0.2rem;
  box-shadow: 0 2px 10px rgba(41,98,255,0.07);
}
.main div[data-testid="stSelectbox"] label {
  font-size: 0.62rem !important;
  font-weight: 600 !important;
  color: #2962FF !important;             /* ACCENT */
  text-transform: uppercase;
  letter-spacing: 0.05em;
}
.main div[data-testid="stSelectbox"] div[data-baseweb="select"] > div {
  min-height: 32px !important;
  font-size: 0.82rem !important;
}
```

Este CSS da al selectbox el aspecto de una tarjeta con borde azul, idéntico en proporción a los KPI cards. El usuario reconoce visualmente que es un elemento de control, no solo texto flotante.

5. Componentes clave

KPI Card

Las KPI cards son HTML generado dinámicamente en Python:

```
def kpi_card(label: str, value: str, sub: str = "") -> str:
    return f"""
    <div class="kpi-card">
      <div class="kpi-label">{label}</div>
      <div class="kpi-value">{value}</div>
```

```

        {"<div class='kpi-sub'>" + sub + "</div>" if sub else ""}
    </div>
    """

```

Se usan con `st.markdown(kpi_card(...), unsafe_allow_html=True)`. El CSS de `.kpi-card` define:

- `border-left: 4px solid ACCENT` — la franja azul característica
- `border-radius: 16px` — esquinas redondeadas
- `animation: fadeUp 0.5s` — aparición con `slide-up`

Type Badge

```

def type_badge(type_name: str) -> str:
    color = TYPE_COLORS.get(type_name, "#90A4AE")
    return f'<span class="type-badge" style="background:{color};">{type_name}</span>'

```

Los type badges son `<span>` con fondo del color del tipo. Se usan inline dentro de tarjetas de Pokémon para mostrar tipo(s) con su color canónico.

Hero Container

El bloque central de Vista General cuando se selecciona un Pokémon destacado:

```

.hero-container {
    background: linear-gradient(135deg, #EEF2FF 0%, #E8F4FD 100%);
    border-radius: 24px;
    min-height: 300px;
    animation: slideIn 0.6s ease both;
}

```

Usa un gradiente suave de azul-lila a azul claro, consonante con los colores de acento. La animación `slideIn` lo hace aparecer desde la izquierda, diferenciándolo visualmente de los elementos que hacen `fadeUp`.

Poke Card (Galería)

```

.poke-card {
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.poke-card:hover {
    transform: translateY(-6px);
    box-shadow: 0 8px 24px rgba(41,98,255,0.15);
}

```

El hover levanta la card 6px y añade una sombra azulada. Es el único elemento con interacción hover en el dashboard — refuerza que la Galería es un explorador, no solo una tabla.

6. Navegación — Sidebar como menú

Estructura del sidebar

```

SIDEBAR
|
├── Logo (Font-pokemon.png como imagen base64)

```

- └─ Título "POKEMON ETL" (Press Start 2P)
- └─ Subtítulo "Portfolio · Oscar Pulido"
- └─ Separador
- └─ Radio buttons → Vista General / Estadísticas / Galería / Comparación / Resumen ETL
- └─ Separador
- └─ FILTROS (se sincronizan con el DataFrame)
 - └─ Tipos seleccionados (multiselect)
 - └─ Rango BST (slider)
 - └─ Power tiers (multiselect)
- └─ Conteo de Pokémon filtrados

Radio buttons como navegación

```
page = st.sidebar.radio("", options=NAV_PAGES, label_visibility="collapsed")
```

Streamlit no tiene un componente `Navigation` nativo. El patrón estándar es usar `st.radio()` en el sidebar. El CSS oculta el indicador de radio y estiliza los labels como ítems de menú:

```
section[data-testid="stSidebar"] div[role="radiogroup"] label {
  padding: 0.5rem 0.75rem;
  border-radius: 10px;
  font-family: 'Press Start 2P', monospace !important;
  font-size: 0.58rem !important;
  color: #555555;
  transition: background 0.15s ease, color 0.15s ease;
}
section[data-testid="stSidebar"] div[role="radiogroup"] label:hover {
  background: rgba(41,98,255,0.08);
  color: #2962FF;
}
section[data-testid="stSidebar"] div[role="radiogroup"] input[type="radio"] {
  display: none; /* Oculta el círculo del radio button */
}
```

El logo en base64

```
_logo_path = Path(__file__).parent / "assets" / "Font-pokemon.png"
with open(_logo_path, "rb") as f:
    logo_b64 = base64.b64encode(f.read()).decode()
st.sidebar.markdown(f'', unsafe_allow_html=True)
```

¿Por qué `base64` y no `st.image()`? `st.image()` es más simple pero no permite control fino de posicionamiento CSS. Al convertir la imagen a `base64` e incrustarla en un `<img>` HTML, se puede aplicar cualquier estilo CSS directamente.

7. Las 5 páginas del dashboard

Vista General (tab_overview)

Propósito: Primera impresión del dataset. KPIs globales + gráficas de distribución + Pokémon destacado.

Elementos:

- **4 KPI cards:** Total Pokémon, BST promedio, tipo más común, Pokémon con mayor BST
- **Filtro burbuja:** Selectbox con CSS de card para seleccionar un Pokémon destacado
- **Hero container:** Sprite oficial + stats del Pokémon seleccionado (o mensaje si se elige "Todos")
- **Gráficas:** Scatter Ataque vs Defensa coloreado por tipo + Distribución de tipos (bar chart)

Estadísticas (tab_stats)

Propósito: Análisis profundo de las distribuciones de stats.

Elementos:

- Histograma de BST con línea de media
- Boxplot de BST por tipo primario
- Heatmap de correlación entre estadísticas
- Top 20 por BST (bar chart horizontal)

Galería (tab_gallery)

Propósito: Explorador visual de todos los Pokémon. Permite buscar y ver la colección.

Elementos:

- Poke cards en grid (4 por fila) con sprite oficial, nombre, tipos y BST
- Las cards respetan los filtros del sidebar
- Hover effect en cada card

Comparación (tab_compare)

Propósito: Comparar dos Pokémon head-to-head con radar chart y detalle de stats.

Elementos:

- Dos selectboxes (Pokémon A vs Pokémon B)
- Radar chart con `go.Scatterpolar` superpuesto (dos trazas, una por Pokémon)
- Tabla de stats con ganador resaltado en cada estadística
- Badge de ganador global (mayor BST)

Resumen ETL (tab_etl)

Propósito: Documentar el pipeline directamente en el dashboard. Demuestra que el candidato entiende la ingeniería detrás del dato.

Elementos:

- Diagrama de flujo Mermaid renderizado como HTML
- Narrative cards: ¿Por qué este proyecto? / ¿Qué hace el pipeline? / ¿Cómo está construido?
- Fases ETL con íconos y descripciones técnicas
- Tabla del dataset (columnas, tipos, descripción)

8. Visualizaciones — Plotly en Streamlit

st.plotly_chart() — La función principal

```
st.plotly_chart(fig, use_container_width=True)
```

`use_container_width=True` hace que la gráfica tome el 100% del ancho disponible. Sin esta opción, Plotly usa su tamaño por defecto (700px fijo).

Plotly Express vs Plotly Graph Objects

	<code>plotly.express` (px)</code>	<code>plotly.graph_objects` (go)</code>
Sintaxis	Una línea	Más verbosa
Cuándo usar	Gráficas estándar	Control fino, gráficas compuestas
Ejemplo	<code>px.scatter(df, x="attack", y="defense")`</code>	<code>go.Scatter(x=..., y=...)`</code>

El dashboard usa ambos: Express para gráficas simples en Vista General y Estadísticas, y Graph Objects para el radar chart de Comparación.

Radar Chart (Comparación)

```
fig = go.Figure()
for poke, color in zip([poke_a, poke_b], [ACCENT, "#E65100"]):
    fig.add_trace(go.Scatterpolar(
        r=stats_values,
        theta=stat_labels,
        fill="toself",
        name=poke["name"],
        line=dict(color=color),
    ))
fig.update_layout(polar=dict(radialaxis=dict(range=[0, max_val])))
```

`go.Scatterpolar` es el tipo de gráfica para radar/spider charts. `fill="toself"` rellena el área bajo la línea, lo que hace visualmente intuitiva la comparación de áreas.

CSS de las gráficas Plotly

```
[data-testid="stPlotlyChart"] {
  background: #FFFFFF;
  border-radius: 20px;
  padding: 0.6rem 0.8rem 0.4rem;
  box-shadow: 0 4px 18px rgba(0,0,0,0.07);
}
```

El contenedor de cada gráfica Plotly recibe una card visual con esquinas redondeadas y sombra. Esto da consistencia al dashboard — todas las gráficas lucen como componentes del mismo sistema.

9. Manejo de datos — Cache y filtros

@st.cache_data — Por qué es crítico

```
@st.cache_data
def load_data() -> pd.DataFrame:
    with sqlite3.connect(DB_PATH) as conn:
        return pd.read_sql("SELECT * FROM pokemon", conn)
```

Sin `@st.cache_data`, cada interacción del usuario (mover un slider, cambiar un filtro) re-ejecutaría la consulta SQL y re-leería el archivo. Con el decorador, Streamlit guarda el resultado en memoria y lo reutiliza — la lectura SQL ocurre una sola vez.

Fallback a CSV:

```
@st.cache_data
```

```
def load_data() -> pd.DataFrame:
    try:
        with sqlite3.connect(DB_PATH) as conn:
            return pd.read_sql("SELECT * FROM pokemon", conn)
    except Exception:
        return pd.read_csv(CSV_PATH)
```

Si `pokemon.db` no existe (el usuario no ejecutó la fase Load), la app carga directamente desde el CSV procesado. El dashboard funciona sin necesidad de ejecutar el ETL completo.

Filtros en sidebar y sincronización con el DataFrame

Los filtros del sidebar devuelven valores que se usan para filtrar el DataFrame antes de pasarlo a cada función de página:

```
tipos = st.sidebar.multiselect("Tipos", options=sorted(df["type_primary"].unique()))
rango_bst = st.sidebar.slider("BST", min_value=0, max_value=int(df["bst"].max()))
tiers = st.sidebar.multiselect("Tier de poder", options=...)

# Filtrado
df_filtered = df.copy()
if tipos:
    df_filtered = df_filtered[df_filtered["type_primary"].isin(tipos)]
df_filtered = df_filtered[df_filtered["bst"].between(*rango_bst)]
if tiers:
    df_filtered = df_filtered[df_filtered["power_tier"].isin(tiers)]
```

Cada función de página recibe `df_filtered`. Las gráficas, KPIs y galería reflejan automáticamente los filtros activos. El DataFrame original `df` se mantiene sin modificar — se usa para el comparador (que debe poder acceder a todos los Pokémon, independientemente de los filtros).

10. Animaciones y micro-interacciones

@keyframes fadeUp

```
@keyframes fadeUp {
    from { opacity: 0; transform: translateY(16px); }
    to   { opacity: 1; transform: translateY(0); }
}
```

Usada en: `.card`, `.kpi-card`, `.poke-card`. Los elementos aparecen desde abajo (16px) con un fade. Crea la sensación de que la página "se construye" al cargarse.

@keyframes slideIn

```
@keyframes slideIn {
    from { opacity: 0; transform: translateX(-20px); }
    to   { opacity: 1; transform: translateX(0); }
}
```

Usada en: `.hero-container`. La entrada desde la izquierda distingue este bloque de los demás — es el protagonista de la Vista General.

Hover en Poke Cards

```
.poke-card:hover {  
  transform: translateY(-6px);  
  box-shadow: 0 8px 24px rgba(41,98,255,0.15);  
}
```

`transition: transform 0.2s ease` hace la animación suave. El color de la sombra (`rgba(41,98,255,0.15)`) coincide con el acento azul — unifica el sistema de diseño.

Transiciones en el menú de navegación

```
section[data-testid="stSidebar"] div[role="radiogroup"] label {  
  transition: background 0.15s ease, color 0.15s ease;  
}
```

`0.15s` es suficiente para sentirse responsivo pero no llamativo. Las transiciones más lentas (`>300ms`) hacen que la UI parezca lenta.

11. Cómo ejecutar y personalizar

Ejecutar el dashboard

```
# Desde la raíz del proyecto  
streamlit run app.py
```

El dashboard se abre automáticamente en `http://localhost:8501`. No requiere ejecutar el pipeline ETL primero — los datos ya están en `data/`.

Configuración de Streamlit (`.streamlit/config.toml`)

```
[server]  
headless = true
```

`headless = true` evita que Streamlit abra el navegador automáticamente en entornos de servidor. En desarrollo local, omitir esto y dejar que Streamlit abra el navegador.

Cambiar la paleta de colores

Los colores se controlan con las constantes al inicio de `app.py`:

```
ACCENT    = "#2962FF"  # ← Cambiar aquí para nuevo color de acento  
BG        = "#F5F7FA"  # ← Cambiar aquí para nuevo fondo  
CARD_BG   = "#FFFFFF"  # ← Cambiar aquí para fondo de cards  
TEXT      = "#111111"  # ← Cambiar aquí para color de texto  
TEXT_SUB  = "#555555"  # ← Cambiar aquí para texto secundario
```

El CSS usa estas constantes mediante f-strings (`f"color: {ACCENT}"`), por lo que el cambio se propaga a todo el dashboard.

Añadir una nueva página

- Crear la función `tab_nueva_pagina(df: pd.DataFrame) -> None`
- Añadir el nombre a `NAV_PAGES`:

```
NAV_PAGES = ["Vista General", "Estadísticas", "Galería", "Comparación", "Resumen ETL", "Nueva Página"]
```

- Añadir el caso al router en `main()`:

```
elif page == "Nueva Página":  
    tab_nueva_pagina(df_filtered)
```

Dependencias necesarias

```
pip install streamlit plotly pandas
```

El dashboard no requiere las dependencias del pipeline ETL (requests, tenacity, loguru) para funcionar.